方法手册导读与补充（白话版）

配合《Stage 1–13 方法手册》（259 页，2026-10-30 版）使用。手册写得很全，但默认读者已经懂术语。这份补充做三件事：用白话解释术语；把每个工具“是怎么想出来、怎么一步步做出来的”讲清楚；把没做、没做成、作废的东西列全，并写好答辩时怎么说。

〇、先看这里：三处结论要降级

读手册时发现，有三处我们之前在 PPT 和讲稿里说得太满了。

1. M3 并不支持 H2（论文身份）

之前的说法：同一篇论文换问法，标题位置余弦 0.95–0.98，换问法移动距离只有换论文的 15–27%，所以标题位置存的是“论文身份”。

手册里的实际结果：预注册补充 2 规定，M3 只看“比 L4 高出多少”，因为 L4 几乎只是文字本身。结果：

- 迁移比 T 在 L16–L40 约 0.92–0.99，但 **L4 就已经是 0.98 / 0.98 / 1.00**；T − T(L4) 在 L16–L40 是 −0.03 到 −0.06，区间不含 0。
- 身份指数 I − I(L4) 在**所有层都小于 0**（L24 −0.011，L48 −0.052）。

意思是：中间层并没有比“只读了文字”的 L4 更“跨问法一致”。三种问法只在标题前 9 个 token 不同，标题完全一样，所以“换问法不变”几乎是构造决定的，不能证明有超出文字的论文身份。手册把这条标为“待复核”（附录 G4 的 V28）。

答辩怎么说：“M3 显示标题位置在换问法时基本不变，但这在 L4 就已经如此，按我们预注册的判据，它不能单独支持‘论文身份’。H2 目前主要靠 M2（标题位置能读出年份、引用数，且 Qwen 明显高于 BERT）和 X1（跨问法替换能恢复作者分数），而 M2 的 Qwen 胜过 BERT 按预注册只作参考。”

要改的材料：第 16 页三个假设表 M3 那列、H2 判断“Holds at the title position”，两份讲稿的 M3 段落。

2. X4 不能排除文字解释（H3）

之前的说法：纯文本基线读不出第一作者，所以 Qwen L8 的 +0.046 不是标题文字带来的。

实际结果：第一版文本基线（kNN）的阳性对照没通过，不作数；第二版在名字首 token 上文本基线不弱于 Qwen。另外 X4 还发现：第一作者 L8 就能解码，年份要到 L40，**和 H2 预测的“先年份后作者”顺序相反。**

答辩怎么说：“X4 的作者信号出现很早，但我们不能排除它来自标题文字，所以 H3 证据不足，仍是开放问题。”

3. X3 也不能证明“论文身份”

结果：改写标题在 L28–L40 比用词相近的近邻 D 更像原标题（最大 +0.16，L36），行为上保留原标题约 90% 的作者优势。但只懂语义的句子向量模型 all-mpnet 在用词匹配的子集上也有相当优势；而且 90% 在 target 和 control 之间一样。表示层面也一样：L36 上 target +0.158、control +0.160。

预注册上要说清楚：按补充 4 的字面判据，X3 其实判为“存在论文身份信号”。文本基线和 target/control 拆分是看到结果后加的，所以属于“与预注册的出入”，但方向是让结论更保守。

答辩怎么说：“X3 说明 Qwen 中间层表示了标题的意思，不只是用词。但不认识作者的语义模型也能做到，认不认得出也没有差别，所以它分不清认出论文和懂得意思。按预注册判据它是通过的，我们事后

加的检验让解读更保守，两者都报告。M3、X3、X4 三个专门的区分实验，都没能证明是认出论文。”

一、术语白话表

1. 模型内部

术语	白话	例子或补充
残差（residual）/ 残差流	模型每一层“手上拿着”的那串数字，每层在上面加一点东西再往下传	Qwen3-32B 每个位置每层是 5,120 个数
Ll / block 输出	第 l 层处理完之后的残差	“L48 的状态”= 第 48 层输出的那 5,120 个数
final norm（RMSNorm）	最后一步变成词之前的缩放，把向量调到统一大小	公式 h / sqrt(平均(h^2) + 1e-6) × 权重
lm_head / unembedding	最后那张“把数字变成每个词的打分”的大表	151,936 行，每行对应词表里一个 token
logit	模型给某个词的原始打分，越高越可能被说出	还没变成概率之前的数
token	模型切词的最小单位	“S. Bree”可能切成 S / . / ␣B / ree
token 角色	按名字结构给 token 分类	initial（单字母）、given（名）、surname（姓）、_first 是词首 token
注意力（attention）	一个位置去看前面其他位置、把信息搬过来	名字位置“看”标题
MLP	只在本位置加工，不看别的位置	常被认为是存知识的地方
o_proj / down_proj	注意力输出、MLP 输出最后那一步线性变换	P2 在这两处之后、加回残差之前替换
bf16 / fp16 / fp32	数字的精度，位数越少越省显存，但舍入误差越大	模型用 bf16 跑；容差问题就来自 bf16 舍入
teacher forcing	不让模型自己生成，而是把正确答案逐个塞进去，看它给正确答案打多少分	测的是“偏好”，不是“会不会写出来”
thinking	Qwen3 先写一段思考再回答的模式	开放回答时关了
attention sink	模型习惯把大量注意力放在开头几个 token 上，那里的状态不代表内容	拟合 lens 时跳过前 16 个位置

2. 提示与位置

术语	白话
作者问法 PROMPT	Who are the authors of the paper titled "{title}" ({year})?\nAuthors: ，裸文本，不套聊天模板
标题最后一个 token（title_last）	标题里最后那个 token。因为模型只能看前面，这时年份还没出现，所以这里不会“偷看”年份
":" 位置（p1）	“Authors:” 的冒号，它的输出就是第一个名字 token
对齐尾段 / tail	真标题和假标题两个提示共同的结尾部分：右引号、年份、“Authors”、“:”，再加上所有名字位置
p1…p9	从提示末尾往前编号：p1 = “:”，p2 = “Authors”，p3 = “)?\n”，p4–p7 = 年份 4 个数字，p8 = “ (”，p9 = 右引号
delete 条件	把标题删掉的提示，测“只有年份时”名字本身好不好猜
单独打分 / 连写打分	单独：每位作者各自接在“Authors:”后；连写：全部作者用逗号接成一串

3. 打分

术语	白话
rank（名次）	1 + 词表里打分比正确 token 高的 token 个数。排第 1 就是模型最想说它
nlr = log10(1/rank)	主分数。第 1 名 0 分，第 10 名 −1，第 100 名 −2，最低约 −5.18
标题特异信号（spec）	真标题分数 − 假标题分数。同一个名字、同样的 token，减完剩下的只能来自“认得这篇论文”
total / topic / prior	把真标题分数拆成三块：prior = 没标题时也有的分；topic = 假标题（同领域）带来的分；spec = 这篇论文特有的分
RR / JMRR	Stage 4–8 用的旧指标：RR = 1/名次；JMRR 是 Jacobian lens 下的平均 RR
7 层积分	Stage 4–8 对 L16/24/32/40/48/56/62 七层取平均
W（金标准胜率）	正确作者首 token 的打分赢过 100 个错误作者首 token 的比例
TA（标题优势）	W(真标题) − 20 个错误标题下 W 的平均
pp	百分点。+2.33 pp 就是胜率高 2.33 个百分点

4. 数据与分组

术语	白话
cohort（9,108 篇）	Dreaming Up Authors 研究的分层样本：从 OAG v2 抽、过滤后，按 8 个领域 × 13 个引用档、每格最多 100 篇取
标注包 / elm	组长用 elm 这套代码在 Qwen3-32B 上做开放回答和多选题、打好标签的数据包；elm 代码和运行记录不在包里

术语	白话
M / H / U、HR2	M 写对的，H 编造的，U 漏掉的；HR2 = (H+U)/(M+H+U)
R0 / R1 / R2	R0 一个都没写对；R1 写对少数（HR2 > 0.5）；R2 基本写对
target / control / excluded / guard	target：写不出但多选题 4 次全对且真选项概率 ≥ 0.5；control：多选最多对 1 次；guard = R2；其余为 excluded
A / B / C / D	target·R0、control·R0、target·R1、control·R1
citation bin（引用档）	按 2 的幂分的 13 档：0–2、3–4、5–8 … 4097+
donor（供体）	把自己的真标题借给别的论文当假标题的那篇
false_0…false_4	每篇的 5 个假标题，false_0 是代价最小的那个，不是随机的
strict 档 / 79%	最严的配对条件；所有（论文, 假标题）配对里 78.8% 来自 strict 档
Pilot50 / Benchmark20	Stage 4–7 的 50 篇和 20 篇小样本
Stage 11 样本	10 个引用档里每档每组最多 60 篇，共 593 + 593 篇
pairs / tune / test	Stage 13 的 1,178 篇配对表，按作者团队分成调参用 596 篇和测试用 582 篇

5. 统计与防泄漏

术语	白话
档内 target − control	只在两组都 ≥ 30 篇的引用档里比，每档权重 n_t·n_c/(n_t+n_c)，两组越平衡权重越大
bootstrap	从样本里有放回地重抽很多次（2,000 次），看结果的波动范围，得到 95% 区间
团队整群 bootstrap	重抽时以“作者团队”为单位整组抽，因为同团队的论文不独立
作者团队（并查集）	共享任一作者全名的论文并成一组，A 和 B 共享、B 和 C 共享，则 A、B、C 一组；全队列 7,247 个团队
5 折交叉验证	分成 5 份，轮流 4 份训练、1 份测试，每篇只在没参与训练的那折上被测
留出预测	某篇论文在它作为测试集时得到的预测
泄漏（leakage）	测试时用到的信息在训练时已经见过，结果会虚高
title reuse（一题两用）	论文 A 的假标题是论文 D 的真标题，D 在训练集，probe 就“见过”这个标题
打乱标签对照	把答案随机打乱再训练，正常应该读不出；读得出就说明方法有问题
阳性对照	一定该有信号的情况，用来证明方法“测得出东西”；比如 “:” 位置读年份（年份就在提示里）

术语	白话
阴性对照	一定不该有信号的情况；比如标点 token、换成别人的作者
预注册 / 补充 1–5	看结果之前写死判定规则的文件，有 sha256 防篡改
Holm 校正	同时做很多个检验时，把显著门槛按顺序调严，防止“碰巧显著”
LOPO（逐对剔除）	每次去掉一对论文重算，看结论是不是靠某一对撑着
R^2	预测解释了多少变化，1 是完美，0 等于只猜平均数
AUROC	随便拿一个正例一个负例，模型把正例排前面的概率；0.5 是瞎猜
准确率	猜对的比例；8 个领域瞎猜约 0.125
MSE	预测误差平方的平均
NULL FLAG	X1 里随机论文 C 的恢复率绝对值超过 0.1 时打的警告

6. 运行与记录

术语	白话
shard	把论文分成的小批，一批一个文件，方便断点续跑
作业号 / sbatch / srun	集群上提交任务的编号和命令
commit	git 的一次存档记录，可以查当时代码是什么样
sha256	文件的“指纹”，文件一改指纹就变
parity 检查	快的算法和慢的算法结果是否一致
tiny 模型测试	用很小的随机模型在 CPU 上把整条流程跑一遍，查代码错误，不看数字

二、工具是怎么做出来的

1. logit lens：最简单的读法

想法：模型最后一层是用 final norm + lm_head 把状态变成词。那把中间某层的状态也直接送进这一步，看看“如果在这里就停下来，模型会说什么”。

做法：lm_head(final_norm(h_l))，然后看正确名字 token 排第几。

缺点：中间层的状态和最后一层“说的是不同的语言”，直接套最后一步会读错。所以中间层几乎全是乱码 token。

2. Jacobian lens：先翻译再读

想法：既然中间层和最后一层“语言不同”，就先学一个翻译：第 l 层的变化会怎样影响 L63？用这个线性关系把中间层状态搬到 L63 的“语言”里，再变成词。

拟合一把 Jacobian lens 的步骤：

1. 准备一批普通文本（比如 WikiText-103 的段落）。
2. 每条文本跑一次前向，存下每层的状态。
3. 对 L63 的 5,120 个维度，每次只在一个维度上放一个“1”，反向传播回第 l 层，得到“L63 第 d 维对第 l 层每个维度有多敏感”，也就是 Jacobian 矩阵 J_l 的第 d 行。5,120 维要反向 5,120 次（实际分批做）。
4. 在这条文本的有效位置上取平均，跳过前 16 个位置（attention sink）和最后一个位置。
5. 对所有文本的 J_l 不加权平均，就得到这一层的 lens。
6. 使用时：lm_head(final_norm(J_l · h_l))。

我们用的四把：

- **public**：Neuronpedia 公开的，557 条 WikiText-103，覆盖 L0–L62。具体用了哪 557 条已按它的代码复原，模型 revision 没有记录。
- **self100**：Stage 3 自己拟合，100 条 WikiText-103，6 层。
- **fitA**：Stage 9 的 Fit A，另外 100 条，7 层。
- **F1 作者 lens（M4）**：Stage 12 用 40 篇“作者问法 + 真作者名单”拟合，专门针对作者任务。

为什么要四把：如果只有一把读不出，可能是这把不好；四把都读不出，才更说明信息不在“词的方向”上。

3. probe 的通用原理

想法：lens 是“现成的翻译”，读不出不代表没有信息。probe 是“专门为这个问题训练的翻译”：给它看几千篇论文的状态和答案，让它自己学怎么从状态读出答案，再拿没见过的论文测它。

为什么用线性、用 ridge：

- 线性：只问“信息能不能用一个简单的加权求和读出来”。如果要很复杂的模型才读得出，那可能是 probe 自己算出来的，不是模型存的。
- ridge：5,120 个权重、只有几千篇论文，很容易死记训练数据。ridge 在目标里加一项惩罚 $\alpha\|W\|^2$，让权重不要太大，减少死记。

每个 probe 都走的六步：

1. 分 5 折（现在都按作者团队分）。
2. 在训练折上标准化：每个维度减均值、除标准差，让 5,120 个维度在同一尺度。
3. 选 α：在一组候选里，用第 0 折的留出误差挑最好的，之后 5 折都用它。
4. 用闭式解算权重：$W = (G + \alpha \cdot \mathrm{tr}(G)/d \cdot I)^{-1} A^T(Y - \text{均值})$，$G = A^TA$。
5. 对测试折做留出预测。
6. 跑对照：打乱标签、换答案、换输入。

4. E4b probe：是怎么想出来的

它要回答的问题：lens 在标题最后一个 token 上读不出作者，那这个位置的状态里到底有没有线性可读的作者信息？

难点：probe 需要“答案”来训练。年份、领域有现成标签，但“作者”是一串名字，没法直接当成一个数字去回归。

解决办法：不直接学“名字”，而是学“模型自己在 ‘:’ 处最后一层的状态”。原因是 “:” 处 L63 的状态，正是模型马上要说出第一个名字 token 时手上的东西，它经过 final norm + lm_head 就直接决定第一个名字。所以：

- 输入 X：标题最后一个 token 在第 l 层的状态（5,120 维）。
- 目标 Y：同一次运行里 “:” 处 L63 的状态（5,120 维）。
- 学到的 probe 就是“从标题位置预测模型最终会在 ‘:’ 处想什么”。

之后怎么打分：

1. 用 probe 对测试论文算出预测的 Y。
2. 把它送进模型自己的 final norm，再只和 4,314 个名字 token 的 lm_head 行相乘（整个队列里所有作者的第一个 token 和所有姓的第一个 token）。
3. 看每位作者**第一个 token** 在这 4,314 个里排第几，取 −log10，再对作者平均。
4. 同一个 probe，输入换成 false_0 那遍（借来的标题）的状态，再打一次分。
5. spec = 真标题分数 − false_0 分数。

参数：层 L16、24、32、40、44、48、52、56、60；α 候选 {0.1, 1, 10}，所有层都选中 1。

对照：

- 换答案：用随机另一篇、同领域另一篇论文的作者去打分，约为 0，说明信号属于这篇论文。
- 只用假标题运行训练：信号相近，说明 probe 学的不是“真标题提示”本身的特殊之处。
- 复核：按作者团队分折、剔除一题两用的论文（剩 1,863 篇，E4b 信号只用其中 target + control 的 1,395 篇）、只在训练折内标准化（两种标准化差别最大 0.0007，结论不变）。

要主动说的局限：

- 只打分第一个 token，而 73% 的作者第一个 token 是首字母，首字母几乎不带标题特异信号。所以它测的是信号最弱的那部分。补充 9（答辩后新增，分析计划在看结果前写定）把这个 probe 扩展到每位作者的全部名字 token 并按角色分开：L40 姓的第一个 token +0.071 [0.058, 0.085]（预注册版 V0：只减 false_0，目标 + 对照 1,395 篇，剔除标题复用，团队聚类 95% CI；381_stage13/reports/e4b_alltok.md），高于首字母 +0.044 [0.030, 0.057]（1,590 篇）；名的第一个 token 最强（L40 +0.124、L56 +0.163（1,706 篇））。原 E4b 只打分第一个 token（多半是首字母），所以低估了标题位置携带的名字信息。
- 只减 false_0，没用 5 个假标题平均。
- 目标是 “:” 处的状态（预测第一作者的首 token），却用来给所有作者的首 token 打分。

5. M2 probe：读论文的其他属性

想法：如果标题位置存的是“这是哪篇论文”，那年份、引用数、领域这些属性也应该读得出来。

做法：

- 输入：标题最后一个 token 的状态，9 层。
- 目标：领域（8 类，one-hot 编码，取最大的那类算准确率）、年份（R^2）、log10(1+引用数)（R^2）、去掉年份后的引用数（先用年份解释掉一部分）、认不认得出（target 记 1、control 记 0，用 ridge 回归再算 AUROC，在每个引用档内算再加权）。
- α 候选 {0.01, 0.1, 1, 10, 100, 1000}。

对照：打乱标签；只用提示里的年份去预测（年份基线）；":" 位置读年份当阳性对照（R^2 0.97–0.98，因为年份就在提示里）；按论文和按团队两种切法比较。

结果（团队切）：年份 R^2 L16 0.38 → L52 0.56；引用数 0.29 → 0.53；领域准确率约 0.70–0.72；认得出 AUROC 0.62 → 0.67。年份基线只能把认得出做到 0.555。

6. M2b / M2c：文本基线

想法：年份、引用数也许光看标题用词就能猜，比如某个领域、某个年代的常用词。所以要一个"没背过这些论文"的模型做下限。

- **M2b TF-IDF**：把标题变成词频向量（1–2 个词的组合），用 ridge 预测。结果年份 R^2 0.10，引用数 0.08。但 TF-IDF 太弱，不能说明问题。
- **M2c BERT**：用 bert-base-uncased 读标题，取每层平均向量，同样用 ridge。结果年份 0.24、引用数 0.19、认得出 0.63。按预注册，BERT 追平 Qwen 才算"风格解释"，Qwen 高出只作参考。

7. M3：同一标题换三种问法

做法：三种问法（问作者、问年份、问领域），标题前都对齐成 9 个 token，存 15 层在标题位置和 ":" 位置的状态。

两个指标：

- **迁移比 T**：在问法 a 上训练 probe，拿去预测问法 b，成绩 ÷ 在 b 上自己训练的成绩。接近 1 说明换问法不影响。
- **身份指数 I**：先把每种问法的向量减均值、变成单位长度；同一篇论文在两种问法下的余弦，减去和别的论文的平均余弦。越大说明"同一篇论文跨问法越像自己"。

为什么要和 L4 比：L4 几乎只编码了文字。如果中间层的 T、I 不比 L4 高，就说明一致性来自"标题文字一样"，而不是模型认出了论文。结果见第〇部分：不比 L4 高。

8. 替换（activation patching）

想法：probe 只能说明"信息在"，不能说明"模型在用"。替换是直接动手：把真标题那遍某处的状态塞进假标题那遍，看作者分数能恢复多少。

通用做法：两遍（真标题干净、假标题损坏），在第 l 层把指定位置的 block 输出替换，其余照常往后算。

恢复率 R = 平均(替换后 − 假) ÷ 平均(真 − 假)，先对所有论文平均再相除。

各个版本：

- **E1**：替换整个对齐尾段（包括名字位置）。
- **E1t**：只替换标题最后一个 token。
- **P1**：一次只换一个位置（p1–p9 或全部名字位置）。
- **P2**：只换某层注意力输出或 MLP 输出。
- **X1**：把"问年份 / 问领域"时的标题状态换进"问作者"的那遍。
- **X6**：E1 换成另外 4 个假标题重做。

9. 阻断（knockout）

做法：从第 l 层到 L63，禁止标题之后所有位置去注意标题 token，所有头都挡。实现方式是在注意力打分上加一个掩码，被挡的格子设成 bf16 能表示的最小值，softmax 后约等于 0。

损失 = 平均(真 − 阻断后) ÷ 平均(真 − 假)。

版本：E2 挡全部；E2n 只挡 ":" 和名字位置；X7 在假标题那遍也做同样阻断，扣掉"被挡本身造成的破坏"。

10. Patchscope（X4）

想法：把论文 A 的状态注入一个只写着占位符 X 的新提示，比如"Paper: X\nThe paper was published in the year"，看模型能不能据此说出 A 的年份。源提示里从来没有年份，所以说得出就只能是"认出了这篇论文"。

做法：源状态取标题最后一个 token 或 ":"，层 8、16、24、32、40、48、56；在目标提示里 X 那个位置的同一层替换；对照用搭档论文 B 的状态。D = 自己 − 搭档。最早可解码层在 tune 上找、在 test 上确认。

11. X4 文本基线

- **第一版（kNN）**：用 TF-IDF 或 BERT 找标题最像的 50 篇论文（排除 A 自己的团队），看 A 第一作者的姓是否出现在里面。**阳性对照失败**，结论作废。
- **第二版（监督）**：从标题文本直接训练分类器预测第一作者的名字首 token 和姓首 token。名字首 token 的阳性对照通过，姓的没通过。修正代码错误后，在名字首 token 上文本基线不弱于 Qwen。

12. X3：改写标题

想法：同一篇论文的标题换个说法，和一篇用词同样相近的别的论文比。模型如果认的是论文，改写版应该更像原版。

几个部件：

- 改写由 Qwen3-32B 生成；和原标题完全相同的 27 条排除。
- **Jaccard**：两个标题共有词数 ÷ 总词数，衡量用词重合度。原版分词丢掉了 3 个字母以下的词，导致 "M22" 这类被漏算，改成 jaccard2。
- **近邻 D**：同领域、无共同姓氏、和原标题 Jaccard 不低于改写版的另一篇论文。
- **NLI**：判断两句话是否互相蕴含的模型，原→改、改→原都要成立。
- **Qwen 判断器**：v1 一致率 7/19，v2 在新 20 条上 0.55，都低于 0.80，不用。改走备用方案：人工核对 + 剔除/不剔除两版敏感性分析。

13. 统计部分

- **档内组差**：只在两组都 ≥ 30 篇的 10 个档里比，按 $n_t \cdot n_c/(n_t+n_c)$ 加权。
- **bootstrap**：Stage 10 起在"档 × 组"内重抽论文 2,000 次；Stage 13 起改为团队整群 bootstrap 为主。
- **作者团队**：共享任一作者全名就并组。注意：分组函数对同样大小的组排序不稳定，换一台机器分折可能不同，点估计最多差 0.006；M2 已把分折存成文件，其他脚本还没查。
- **多重比较**：Stage 10–13 没有做校正，规则是"所有层都报告、区间不含 0 才算"。Stage 8 补做了 Holm，只有 L62 通过。

三、没做、没做成、作废的，以及答辩怎么说

Stage 3、7、9

项目	现状	答辩怎么说
Stage 3 public lens 前两次验证运行	显存不够、设备映射报错，由修复版取代	“前两次因技术问题失败，用修复后的运行”
Stage 3 split-half 稳定性检验	被 Stage 9 设计取代，但 Stage 9 没做完	“原本由 Stage 9 接替，Stage 9 未完成，所以 lens 稳定性只有 Fit A 一份证据”
Stage 7.1B 终端读出	只做了 1,166 / 15,519，约 7.5%，没估计任何效应	“这部分只跑了一小部分，没有结论，不和 7.1 合并”
Stage 7.2、7.3 部分区间	原始输出里没有，已重算替换	“原区间无出处，用可复现的重算值，原值都在重算范围内”
Stage 9 Fit B	停在 66/100；已列入重做计划（补充 12 第 2-1 项，批次 2）	“Fit B 未完成，未使用；Stage 10 只用 Fit A”
Stage 9B 留出集比较、9C 影响因素分析	9B 列入重做计划（补充 12 第 2-1 项，批次 2）	“列入重做计划，尚未运行”

Stage 8

项目	现状	答辩怎么说
主终点	分析计划在 10-18 14:29 写定锁定（主终点为 7 层积分），同日 21:14 的 PROTOCOL 改为 L56，两者都早于 8C 打分；改动人和理由待查（U28）	“冻结计划的主终点是 7 层积分，结果 +2.33 pp [+1.50, +3.19]；L56 两种都报告”
Holm 校正、逐对剔除	计划里有、当时没做，10-30 补做	“补做后，逐层只有 L62 通过 Holm；积分 A−B 逐对剔除 100 次都不含 0，L56 有 14 次跨 0”
8E 外部审计	原脚本找不到，已从原始数据独立重算	“点估计与报告相同”

Stage 10–11

项目	现状	答辩怎么说
原主指标 Σ1/rank	改成 log10(1/rank)	“1/rank 只看前几名，改用对数名次；原始名次都保存了”
parity 容差 0.25 → 0.5	看到 bf16 差距后放宽	“这是事后修订，原 FAIL 记录保留，差距来自 bf16 舍入”
E1t 基线	批量舍入导致原基线不可用，改用同批基线	如实说

项目	现状	答辩怎么说
E1 批量舍入噪声	没专门量化	“估计约 0.01，远小于主效应”
E4b 原版结论	被复核版取代	“复核后数值上‘目标组更高’仍成立”
“只用假标题训练信号更强”	作废	“来自泄漏，复核后不再更强，已撤回”

Stage 12

项目	现状	答辩怎么说
P1 首次运行	崩溃，修复后重算	如实说
V100 路径	写好但没用	“显存不够，所有计算都在 A100-80GB 上”
N2 中间层读出	数据已存，只分析了最后一层	“只报告了输出层”
F1 同名作者	不重拟合，评估时剔除 3 篇	如实说
P2 显式检查	没有“替换 L63 应完全恢复”的检查，已决定补 1 个分片	“补做中”
F1 对半拟合	不做	“写进局限”

Stage 13

项目	现状	答辩怎么说
M2 按论文切的 false_0 结果	作废，按团队切为准	“假标题复用泄漏，已作废”
M2 “:” 位置年份	降为阳性对照	“那里看得到年份，只用来证明方法有效”
M2c 原判据	改为单向	“BERT 追上才作数，Qwen 胜出只作参考”
M3 判据	待复核：中间层不高于 L4	见第〇部分
M3 年份、作者首 token 分数	已存未用	“计划外数据，未分析”
M5	能用已有数据回答的部分 2026-11-05 已在 CPU 上做完；名字位置的 probe / patchscope 列入补充 11、12	“CPU 部分已完成，GPU 部分在重做计划里”

项目	现状	答辩怎么说
X1 随机论文 C	只打了 NULL FLAG，没相对 C 重算	“C 的恢复率超过 0.1 时有警告，作者减 C 在各层都至少 +0.30”
X2 只替换子空间	2026-11-05 已完成（补充 6）：H3 按规则通过、H2 不支持，但作者子空间几乎解码不出作者，搬动量只有完整替换的约 4% (381_stage13/reports/x2_report.md)	“X2 不能作为 H3 的有力证据”
X3 正式运行	已完成，结论否定：L28–L40 改写版比近邻 D 更像原标题（最大 +0.16），行为保留约 90%；但句子向量模型 all-mpnet 在匹配子集上也有相当优势，行为恢复 target 0.90 与 control 0.91 无差别	“X3 说明 Qwen 中间层表示了标题的意思，不只是用词，但分不清是认出论文还是懂得意思，所以不能支持 H2”
X3 判断器 v1、v2	都不用	“一致率低于 0.80，改用人工核对加敏感性分析”
X4 文本基线第一版	阳性对照失败，作废	见第〇部分
X4 按 token 角色报告	X4 做了，X6、X7 没做	“X6、X7 只报告了整体”
X5 层序量化	没做，等 M3、X4 后用已有数据做	“尚未完成”
group_folds 不稳定	只修了 M2	“其他脚本在核查”

四、建议让 Claude Code 补的

1. M3 报告的自动判定语句改成按补充 2 的规则（看 T − T(L4)、I − I(L4)），并把 PPT 第 16 页的 H2 判断改为“M3 不支持，H2 主要依据 M2、X1”。
2. X1 的 year/field 恢复率相对随机论文 C 重算。
3. X6、X7 按 token 角色补报告。
4. 所有用 group_folds 的脚本检查分折是否一致，统一读存档文件。